

2025 전기 졸업과제 착수보고서

ZNS SSD를 활용한 키-밸류 스토어의 성능 개선 연구



202055524 김의현

202055532 나도운

202055545 박정민

지도교수 안성용

팀 NUMPY

하드웨어/보안 분과(D)

차례

1 연구 개요	1
2 연구 배경	1
2.1 키-밸류 스토어, RocksDB	1
2.2 ZNS SSD	2
3 연구 동기	5
3.1 BlobDB와 Wisckey의 차이점 분석	5
3.2 단순한 Victim Selection	6
3.3 부적절한 라이프타임 힌트 부여	9
4 추진 방안 및 제약사항 검토	9
5 일정 및 역할 분담	9
참고 문헌	11

1 연구 개요

본 과제는 ZNS(Zoned Namespace) SSD를 활용하여 키-밸류 스토어인 RocksDB의 성능 개선을 목표로 한다. RocksDB의 Integrated BlobDB와 ZNS간의 연동 문제를 중심으로, GC 전략과 잘못된 라이프타임 힌트가 일으키는 문제를 실증하고 개선하는 것이 그 목표이다. 결과적으로 보다 ZNS와 잘 통합된, 불필요한 데이터 복사가 줄어든, 보다 나아진 BlobDB를 만들고자 한다.

2 연구 배경

2.1 키-밸류 스토어, RocksDB

배경

오늘날 많은 어플리케이션은 계산 중심이 아닌 데이터 중심적이다 [1]. 다양한 데이터를 다룰 일이 점점 증가하는 추세이며, 이에 따라 관계형 모델을 채택하지 않은 DB의 수요도 늘고 있다. 이러한 DB를 NoSQL이라 부르기도 하며, 키-밸류 스토어도 여기에 속한다.

특히 LSM-Tree(Log-Structured Merge Tree) 구조의 키-밸류 스토어를 많이 활용한다. 이러한 키-밸류 스토어는 특히 쓰기 위주의 워크로드에 강하며, 랜덤 쓰기를 순차 쓰기로 전환하는 효과가 있기 때문에 SSD와 상호운용 시 여러 이점이 있다. 대표적인 키-밸류 스토어로 RocksDB [2], LevelDB가 있다.

이번 졸업과제에서 다룰 키-밸류 스토어는 RocksDB이다. 선정한 이유는 다음과 같다. 이미 다방면으로 연구되었고 그만큼 많은 연구를 반영한 상태이다. 또한 LSM-Tree 구조인 만큼 SSD와 잘 어울리며, 결정적으로 ZNS(Zoned Namespace SSD)를 지원하기 때문이다.

RocksDB의 구조

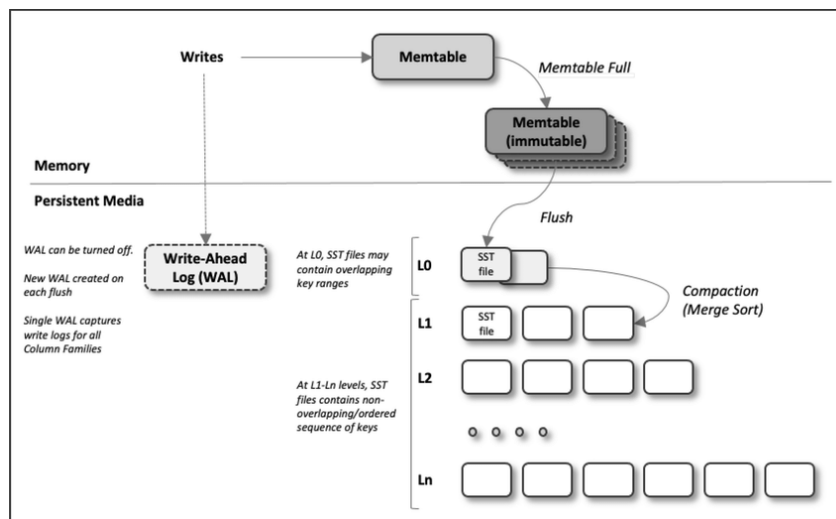


그림 1: LSM-Tree Overview. (출처: [2])

이제 RocksDB의 내부를 알아보자. RocksDB는 LSM-Tree구조이다. 그림 1에서 알 수 있듯이, 크게 in-memory 자료구조와 in-disk 자료구조로 나눌 수 있다. in-memory 자료구조는 여러 개의 Memtable로 구성되며, in-disk 자료구조는 여러 레벨의 SST(Sorted String Table) 파일로 구성된다.

RocksDB의 Memtable은 키-밸류 쌍의 skip list이다. 모든 입력은 Memtable에 먼저 저장되며, 일정 크기에 도달하면 Immutable Memtable이 된다. 이 Immutable Memtable은 차후에 디스크의 L_0 으로 flush된다.

SST 파일은 키-밸류 쌍을 저장한 파일이다. 키 기준으로 정렬되어있으며, 레벨 별로 관리된다. 레벨이 특정한 조건을 만족하는 경우, 해당 레벨의 파일을 아래 레벨로 옮긴다. 겹치는 범위의 키를 가지는 SST 파일을 찾은 뒤 병합하는 것이다. 이를 Compaction이라 한다.

이러한 구조는 여러 장점을 가진다. B+Tree의 경우 모든 업데이트가 즉시 디스크에 반영되는 반면, LSM Tree는 메모리 상의 자료구조에만 반영하다가 어느정도 모이고 나서야 디스크에 쓴다(batched write). 쓰기 위주의 워크로드에 강한 이유이다.

무엇보다 중요한 점은, 순차 쓰기만이 일어난다는 점이다. 메모리 상에서 한번 정렬한 뒤 쓰기 때문에, 위의 모든 과정에서 순차 쓰기만이 일어난다. SSD는 내부적으로 in-place update를 허용하지 않고 out-of-place update만 허용하기 때문에, 순차 쓰기 위주의 워크로드에서 성능상 이점이 있다. 더 나아가 out-of-place 제약을 노출하는, ZNS 특유의 인터페이스와도 호환이 잘 되어 ZNS 지원을 쉽게 도입하였다 [6].

Wisckey: Value 분리하기

LSM-Tree의 Compaction은 성능에 많은 영향을 미친다. Compaction 과정에서 I/O가 아주 많이 발생하기 때문이다. 실제로 다양한 Compaction 전략이 연구되었다.

Wisckey는 LSM-Tree의 크기를 줄여 Compaction 시 이동하는 데이터의 양을 줄인 연구이다 [3]. 밸류가 상당히 크다고 가정하자. 밸류를 별도의 로그에 저장하고, LSM-Tree에는 밸류의 위치만을 저장하면 트리의 크기가 줄어들 것이다. 따라서 Compaction 시 이동하는 데이터를 줄일 수 있게 된다.

Wisckey의 밸류 분리 테크닉은 RocksDB에도 도입되었으며, 이를 BlobDB라고 한다 [4]. 이번 연구에서는 BlobDB를 다루고자 한다. RocksDB의 다른 부분에 상대적으로 연구가 덜 이루어졌으며, Wisckey와 다르게 구현한 부분도 발견하였다. 그 밖에도 단순히 오래된 파일부터 정리하는 식으로 Blob 파일을 관리하는 등 [5], 전반적으로 고도화가 덜 되었다고 판단하였다. 자세한 판단 근거는 3에서 다룬다.

2.2 ZNS SSD

ZNS 개요

기존 SSD는 블록 인터페이스를 기반으로 데이터를 자유롭게 읽고 쓰도록 설계되어 있다. 이는 하드디스크에서 파생된 모델로, 논리적 블록 주소(LBA, Logical Block Address)를 단순한 배열처럼 다루기 때문에 응용 프로그램 개발에는 유리하다. 그러나 플래시 메모리는 특성상 데이터를 덮어쓰려면 먼저 블록 단위로 지워야 하므로 SSD 내부에 FTL(Flash Translation Layer)이 추가로 필요하다. 이로 인해 많은 DRAM과

over-provisioning 영역이 요구되며 성능 저하를 유발하는 garbage collection이 발생한다. 이러한 구조적 비효율을 흔히 ‘Block Interface Tax’라 부른다 [6].

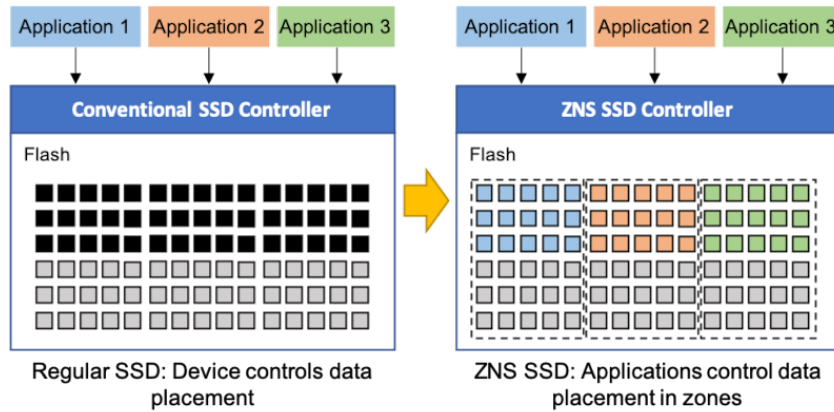


그림 2: 기존 SSD와 ZNS 비교. (출처: [7])

이를 해결하기 위해 제안된 ZNS 기술은 SSD의 LBA 공간을 여러 개의 zone으로 나누고 각 zone에 대해 순차적 쓰기만 허용하는 새로운 인터페이스이다. 그림 2는 기존 SSD와 ZNS SSD의 내부 구조를 비교한 것이다. 그림에서 볼 수 있듯 기존 SSD에서는 애플리케이션의 쓰기 요청이 FTL에 의해 장치 전체에 비순차적으로 배치되지만 ZNS SSD에서는 각 애플리케이션이 개별 zone을 할당받아 데이터가 논리적으로 분리되고 순차적으로 쓰이도록 강제된다. 이를 통해 내부 GC를 제거하고 쓰기 증폭(write amplification)을 최소화하며 SSD의 용량과 수명을 향상시킬 수 있다.

ZNS의 쓰기 제약과 Zone 동작 원리

ZNS SSD는 각 zone에 대해 쓰기 순서를 엄격히 제한한다. 하나의 zone은 처음에 EMPTY 상태로 시작하며, 쓰기가 시작되면 OPEN 상태로 전이된다. 이후 쓰기가 끝나면 zone은 CLOSED 또는 FULL 상태가 되며 추가 쓰기를 위해서는 반드시 explicit reset 명령을 통해 초기화되어야 한다. 이때 호스트는 각 zone에 대해 현재 쓰기 가능한 위치를 나타내는 write pointer를 관리해야 하며 만약 sequential write 조건을 만족하지 않으면 에러를 반환한다.

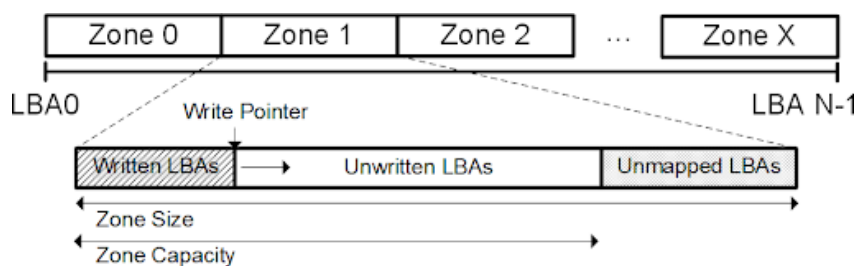


그림 3: Zone의 구성 방식. (출처: [6])

그림 3은 ZNS SSD에서 zone이 어떻게 구성되는지를 보여준다. 각 zone은 일정한 크기의 논리 주소 공간을 가진다. 각 논리 주소 공간은 쓰여진 영역(Written LBAs)과 쓰이지 않은 영역(Unwritten LBAs)으로 구분되고 write가 발생하면 write pointer가 점진적으로 증가한다. 이러한 구조는 호스트가 명시적으로 데이터를 배치하고 블록 지우기 단위를 직접 고려하게끔 해 기존 SSD에서 SSD 내부에 숨겨져 있던 관리 작업을 호스트 측에서 명확하게 제어할 수 있도록 만든다.

ZNS의 성능

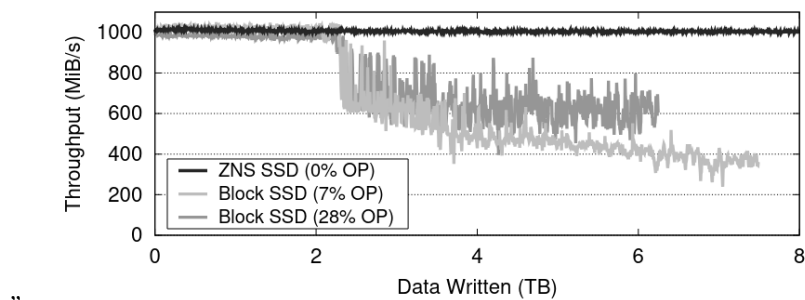


그림 4: ZNS와 일반 SSD간의 데이터 쓰기 성능 비교. (출처: [6])

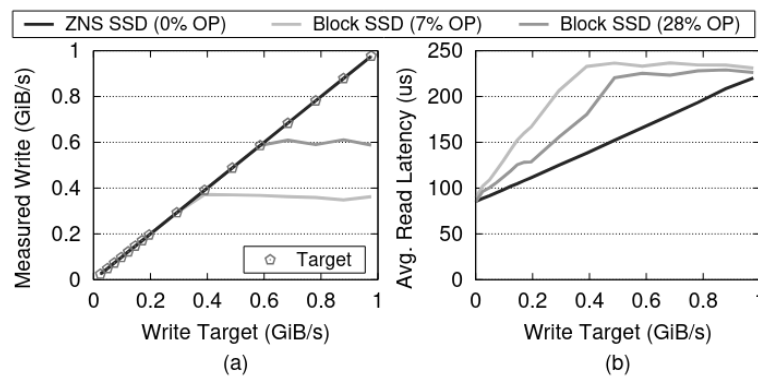


그림 5: 쓰기 부하 증가에 따른 ZNS SSD와 SSD의 비교. (출처: [6])

ZNS는 단순히 인터페이스를 바꾸는 수준을 넘어 실질적으로 SSD의 성능과 안정성을 대폭 향상시키는 효과를 보인다. ZNS의 가장 큰 이점 중 하나는 호스트 측에서 zone 단위로 데이터를 순차적으로 쓰고 직접 GC를 제어함으로써 SSD 내부의 write amplification과 GC overhead를 제거할 수 있다는 점이다. 이를 입증하기 위해 동일한 하드웨어 기반에서 Block Interface SSD와 ZNS SSD의 성능을 비교한 실험이 수행되었다.

그림 4는 총 2TB의 데이터를 4회 반복해서 쓰는 워크로드에서의 sustained write throughput을 보여준다. Block SSD는 2TB를 초과한 시점부터 내부 GC가 발동되며 throughput이 급격히 하락하는 반면 ZNS

SSD는 GC 없이 일정한 쓰기 속도를 유지한다. 이는 ZNS SSD가 별도의 over-provisioning 없이도 안정적인 쓰기 처리 성능을 유지할 수 있다는 것을 시사한다.

또한 ZNS는 쓰기 중 발생하는 읽기 요청에 대해서도 낮은 tail latency를 보장한다. 그림 5는 쓰기 속도를 단계적으로 증가시키면서 랜덤 읽기 성능을 측정한 결과이다. 여기서 ZNS는 block SSD에 비해 최대 64% 낮은 평균 read latency를 기록하였다. 이처럼 ZNS는 내부 GC로 인한 간헐적 성능 저하 없이 일관된 read/write 성능을 제공한다.

ZenFS: RocksDB의 ZNS 지원

RocksDB는 ZenFS라는 플러그인을 통해 ZNS를 지원한다 [6]. RocksDB의 파일시스템 인터페이스 하에 구현된 유저레벨 파일시스템이다. 유사한 라이프타임을 가지는 파일끼리 같은 Zone에 모이도록 Zone을 할당한다. 달리 말하면 라이프타임 힌트에 크게 의존한다.

BlobDB는 ZenFS를 고려하지 않고 작동한다. 그런데 ZenFS는 파일의 라이프타임 힌트에 크게 의존한다. 그런데 BlobDB는 Blob 파일의 라이프타임 힌트를 제대로 부여하지 않는다 [5]. 이번 연구에서는 이 부분을 주목하고자 한다.

3 연구 동기

3.1 BlobDB와 Wiskey의 차이점 분석

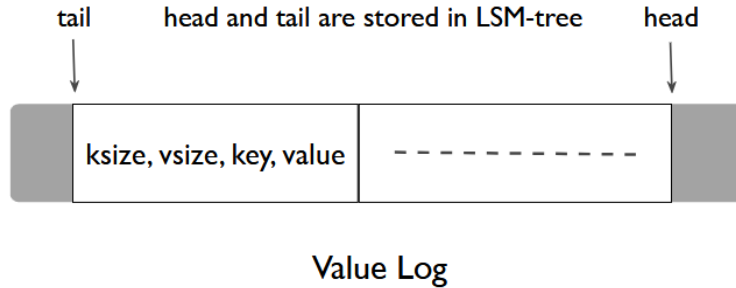


그림 6: Wiskey의 vLog 구조. (출처: [3])

Algorithm 1 (단순화된) vLog GC

```

1: for all  $key, value, value\_pos \in victim\_vLog\_region$  do
2:   if  $queryValuePos(LSM, key) \neq value\_pos$  then
3:      $Insert(LSM, key, value)$ 
4:   end if
5: end for
6:  $remvoeFromDisk(victim\_vLog\_region)$ 

```

표 1: 가상머신 환경

코어 수	4
메모리	16GB
커널 버전	Linux Kernel 6.1.0-34
Zone 크기	64MB
Zone 개수	64
Zone 페이지 크기	16KB
SSD 총 용량	4GB

앞서 말했듯 Wisckey는 밸류를 별도의 로그(이하 vLog)에 분리하여 저장한다. 특이하게도 vLog에 밸류 뿐만 아니라 키를 같이 저장한다.(그림 6 참조.) LSM-Tree와 vLog를 별도로 관리하기 위함이다.

LSM-Tree가 Compaction을 통해 유효하지 않은 항목을 제거하듯이, vLog도 garbage collection(이하 GC)를 통해 유효하지 않은 밸류를 털어낸다. 알고리즘 1은 Wisckey의 vLog GC 전략을 정리하여 의사 코드 형태로 기술한 것이다 [3]. victim의 모든 원소를 순회하면서 유효성을 확인하고, 유효하지 않은 데이터는 트리에 다시 삽입한다. 그리고 나서 해당 영역을 삭제한다. 이 과정에서 키가 필요하기 때문에 vLog에도 키를 저장한다.

그러나 BlobDB는 Wisckey와 사뭇 다르게 동작한다. BlobDB에서는 Blob 파일이 Wisckey의 vLog 역할을 하며 키를 중복해서 저장하지 않는다. 따라서, Wisckey처럼 Blob 파일을 대상으로 하는 GC를 별도로 수행할 수 없다.

결국 BlobDB에서는 LSM-Tree Compaction과 Blob 파일 GC가 동시에 일어난다. Compaction 시 SST 파일과 연결된 Blob 파일을 추적하여, GC 대상인지 확인한다. GC 대상이라면 유효한 데이터만을 옮긴다.

이렇듯 Wisckey와 BlobDB간의 차이점을 식별하였다. 지금으로선 Wisckey와 BlobDB 중 어느 쪽이 더 나은지 알 수 없다. 명확한 판단을 위해서는 다양한 실험이 필요하다.

3.2 단순한 Victim Selection

BlobDB의 victim selection 전략은 아주 단순하다. 전체 Blob 파일 중 정해진 비율만큼을 오래된 순서대로 고른다. (이하 oldest-first policy) 데이터를 라이프타임 별로 구별해 저장하거나 하지 않는다.

단순한 전략이 항상 잘못된 것은 아니다. 잘 작동한다면야 단순할수록 좋다. 그러나 실제 상황에서 적절히 작동하지 않는다면 비효율이 된다. 여기서는 그 점을 검증해보려 한다.

그리하여 oldest-first policy의 기본 가정을 검증할 수 있는 실험을 수행하였다. 정말 오래된 파일일수록 유효하지 않은 데이터가 많은지 확인하는 것이다. 그리하여 매 Compaction 직전마다 Blob 파일들의 상태를 확인해보기로 하였다.

다음은 실험환경 설명이다. 가상머신으로 ConfZNS를 사용하였다 [8]. 표 1은 ConfZNS로 구성한 환경을

표 2: db_bench 설정

workload	mixgraph
num	2000000
i/o type	direct i/o
blob gc force threshold	1.0
compaction style	leveled compaction

표 3: mixgraph 옵션

key_dist_a	0.002312
key_dist_b	0.3467
keyrange_dist_a	14.18
keyrange_dist_b	2.917
keyrange_dist_c	0.0164
keyrange_dist_d	-0.08082
keyrange_num	30
value_k	0.2615
value_sigma	25.45
value_theta	1024
mix_max_value_size	8196
put ratio	1.0
get ratio	0
iter ratio	0

정리한 것이다.

db_bench로 생성한 워크로드 하에서 실험을 진행하였다. 표 2는 db_bench 설정이다. 현실과 가까운 워크로드를 생성하기 위해 mixgraph를 사용하였다 [9]. 표 3은 mixgraph 옵션을 따로 정리한 것이다. 지금은 쓰기에만 관심이 있으므로, put_ratio를 1로 두어 쓰기만 발생하도록 조정하였다.

실험 결과는 다음과 같다. 워크로드 수행 도중 총 8번의 Compaction이 있었으며 파일이 오래될수록 죽은 데이터가 증가하는 경향을 보인다. 그러나 그림 7을 보면, 이러한 경향을 따르지 않는 파일도 있다.

데이터마다 라이프타임이 다르기 때문에 이런 일이 생겼다고 생각한다. mixgraph는 key space의 특정 구간에서 I/O가 쏠려있다 [9]. 자주 입·출력을 수행하는 구간의 데이터는 짧은 라이프타임을 가지고, 그렇지 않은 구간의 데이터는 긴 라이프타임을 가진다. 따라서 Blob 파일이 오래되어도 죽은 데이터가 적을 수 있다. 물론 확신을 가지려면 더 다양한 환경에서 실험해야 할 것이다.

이번 절에서는 특정 워크로드 하에서 oldest-first policy 정책의 한계를 확인하였다. 일반화된 결론을 내리기에는 실험 범위가 제한적이므로 향후 다양한 조건에서 재검증이 필요하다. 그러나 만약 가설이 맞다면, 밸류를 라이프타임별로 분리하여 성능 개선을 노릴 수 있다.

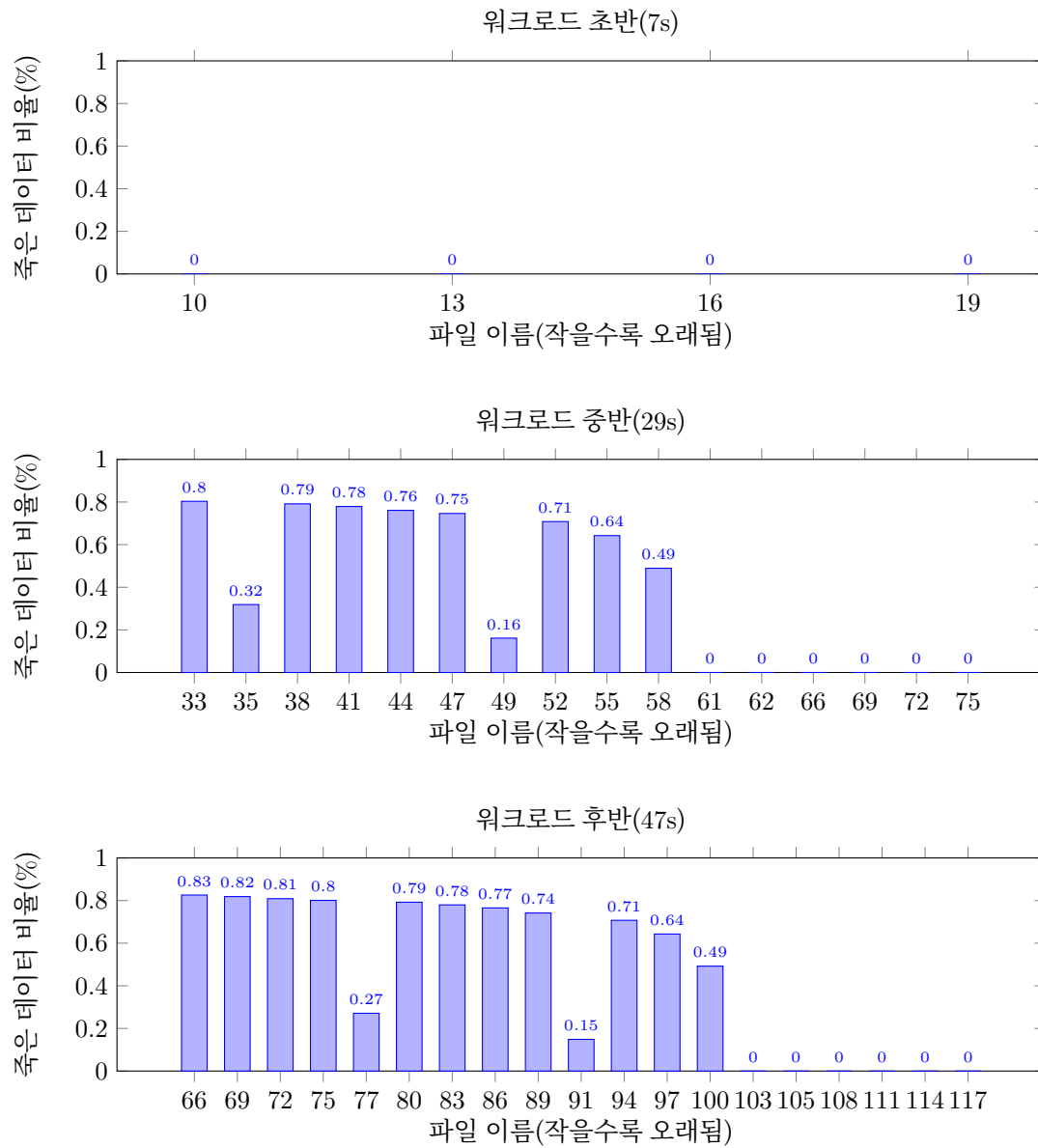


그림 7: 초반 / 중반 / 후반 시점의 죽은 데이터

3.3 부적절한 라이프타임 힌트 부여

BlobDB는 Blob 파일에 적합한 라이프타임 힌트를 부여하지 않는다. 코드를 분석해본 결과 Blob 파일의 라이프타임을 추정하는 로직이 없었으며, 그저 SST 파일의 라이프타임 힌트를 재활용하는 것으로 보인다. 앞서 말했듯, 부적절한 라이프타임 힌트 부여는 바람직하지 않으며 ZenFS와 운용할 때 문제를 일으킨다.

성능 개선을 위해서는 Blob 파일에 적절한 라이프타임 힌트를 부여하여야 한다. 그러나 3.2에서 이야기하였듯이 현행 BlobDB는 밸류를 어떤 식으로든 분류하지 않으며, 단순히 오래된 파일에 유효하지 않은 데이터가 많을 것이라 가정하고 동작한다.(oldest-first policy) 따라서 라이프타임을 추정하기 어렵다.

Same Victim GC는 ZenFS가 Blob 파일의 라이프타임 힌트를 무시하도록 고쳤다. 대신 oldest-first policy에 맞게 Zone을 관리하여 ZenFS의 쓰기증폭을 대폭 줄였다 [5]. 단, oldest-first policy에 국한되는 것이 그 한계이다.

당연히 BlobDB에서 정상적인 라이프타임 힌트를 주는 것이 바람직하지만, 지금의 정책 하에서도 가능할지 모르겠다. 혹시나 BlobDB의 정책을 바꾼다면 그에 맞는 고민이 필요하다. 여러모로 생각해 볼 여지가 많다.

4 추진 방안 및 제약사항 검토

BlobDB의 ZNS 지원 개선이 연구의 궁극적인 목표이다. 따라서 다양한 실험을 통해 앞서 식별한 문제들이 BlobDB에 어떤 영향을 끼치는지 알아내야 한다. 이를 바탕으로 BlobDB를 개선하여 불필요한 데이터 복사를 줄이도록 한다. 그리고 개선 정도와 한계, 의의 등을 정리할 것이다.

대부분의 실험은 ConfZNS 위에서 이루어질 것이다. 에뮬레이터가 갖는 특성과 한계를 명확히 인지하고 실험을 진행하여야 한다. ConfZNS가 아직 성숙하지 않은 편이기에 각별히 주의해야 한다.

이를 위해 정기적인 팀 회의와 다양한 스터디를 가질 것이다. 코드 분석 스터디, 논문 스터디를 통해 전반적인 이해도를 높이고 팀 회의를 통해 진행상황을 관리하고 업무를 배분할 것이다. 상호 검토와 협력, 건전한 비판으로 목표를 달성하도록 한다.

5 일정 및 역할 분담

그림 8과 표 4는 개발 일정과 역할 분담이다. 연구의 불확실성을 감안하여 일정을 넉넉하게 잡았다. 변동이 생기는 경우 추후 보고서에 반영할 예정이다.

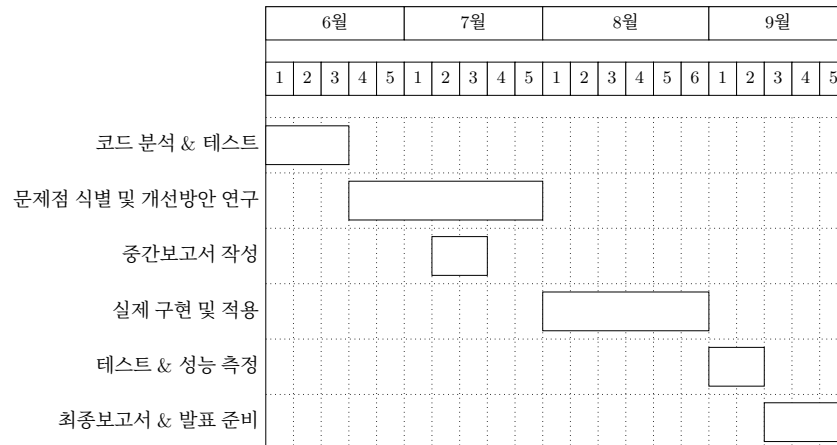


그림 8: 연구 일정 개요

표 4: 역할 분담 개요

이름	역할
김의현	BlobDB 분석 BlobDB 개선을 위한 논문 스터디 성능 평가 및 결과 분석
나도운	BlobDB 분석 BlobDB 하에서 zenfs 성능 분석 성능 평가 및 결과 분석
박정민	BlobDB 분석 BlobDB 파일 GC 개선점 탐색 성능 평가 및 결과 분석

참고 문헌

- [1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, English, 1st edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly Media, Apr. 2017, p. 3, ISBN: 978-1-4493-7332-0.
- [2] *RocksDB-Overview*. Accessed: May 13, 2025. [Online]. Available: <https://github.com/facebook/rocksdb/wiki/RocksDB-Overview>.
- [3] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "WiscKey: Separating Keys from Values in SSD-Conscious Storage," *ACM Trans. Storage*, vol. 13, no. 1, 5:1–5:28, 2017, ISSN: 1553-3077. DOI: [10.1145/3033273](https://doi.org/10.1145/3033273).
- [4] L. Tamasi, *Integrated blobdb*, May 2021. Accessed: May 13, 2025. [Online]. Available: <https://rocksdb.org/blog/2021/05/26/integrated-blob-db.html>.
- [5] H. Hwangbo et al., "Towards A Unified Garbage Collection Strategy in ZNS Key-Value Store File Systems Using Same-Victim GC," in *2024 32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2024, pp. 1–8. DOI: [10.1109/MASCOTS64422.2024.10786550](https://doi.org/10.1109/MASCOTS64422.2024.10786550).
- [6] M. Bjørling et al., "ZNS: Avoiding the block interface tax for flash-based SSDs," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, USENIX Association, 2021, pp. 689–703, ISBN: 978-1-939133-23-6.
- [7] *SSDs with NVMe Zoned Namespace (ZNS) Support*. Accessed: May 13, 2025. [Online]. Available: <https://zonedstorage.io/docs/introduction/zns>.
- [8] I. Song et al., "ConfZNS : A Novel Emulator for Exploring Design Space of ZNS SSDs," *Proceedings of the 16th ACM International Conference on Systems and Storage*, ACM Conferences, pp. 71–82, Jun. 2023, ISSN: 9781450399623. DOI: [10.1145/3579370.3594772](https://doi.org/10.1145/3579370.3594772).
- [9] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du, "Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook," en, 2020, pp. 209–223, ISBN: 978-1-939133-12-0.